Sometimes T&& is not an rvalue reference. It binds to anything.

```
void f(std::string&& s);

template <class T>
void g(T&& x);
```

`f` takes only temporaries. `g` takes **anything**.

swipe →

When is it a forwarding reference?

When `T` is **deduced** and the type is exactly `T&&`.

DECLARATION	KIND
`void f(std::string&&)`	rvalue ref
`template <class T> void g(T&&)`	**forwarding**
`auto&& x = e;`	**forwarding**
`template <class T> void h(std::vector<T>&&)`	rvalue ref

The last one is not plain `T&&`, so nothing is forwarded.

What T becomes

```
template <class T>
void g(T&& x);

int n = 1;
g(n);
g(5);
```

CALL	T	X IS
`g(n)`	`int&`	`int&`
`g(5)`	`int`	`int&&`

An lvalue makes `T` a reference, and `&&` collapses to `&`. An rvalue leaves it a plain type.

The catch: x has a name

```
void f(std::string&);    // A
void f(std::string&&);   // B

template <class T>
void wrap(T&& x) {
    f(x);
}

wrap(std::string("hi"));
```

It calls **A**, not **B**.

Inside `wrap`, `x` is a named object, so it is an lvalue. The fact that it came from a temporary is lost.

The fix: `std::forward`

```
template <class T>
void wrap(T&& x) {
    f(std::forward<T>(x));
}
```

YOU PASS	F GETS
an lvalue	an lvalue (**A**)
an rvalue	an rvalue (**B**)

`std::forward` passes on what it got.

`std::move` always casts to an rvalue. `std::forward<T>` casts only if the argument was one.

Where you see it

```
template <class T, class... Args>
std::unique_ptr<T> make(Args&&... args) {
    return std::unique_ptr<T>(
        new T(std::forward<Args>(args)...));
}
```

Every argument reaches the constructor **as it was passed**: copied if it was an lvalue, moved if it was a temporary.

This is the pattern behind `std::make_unique` and `emplace_back`.

Which one do you mix up: move or forward?